

Parallelizing TC rules update

Jul 2018

© 2018 Mellanox Technologies

Problem statement

Who needs to create millions of TC rules anyway?

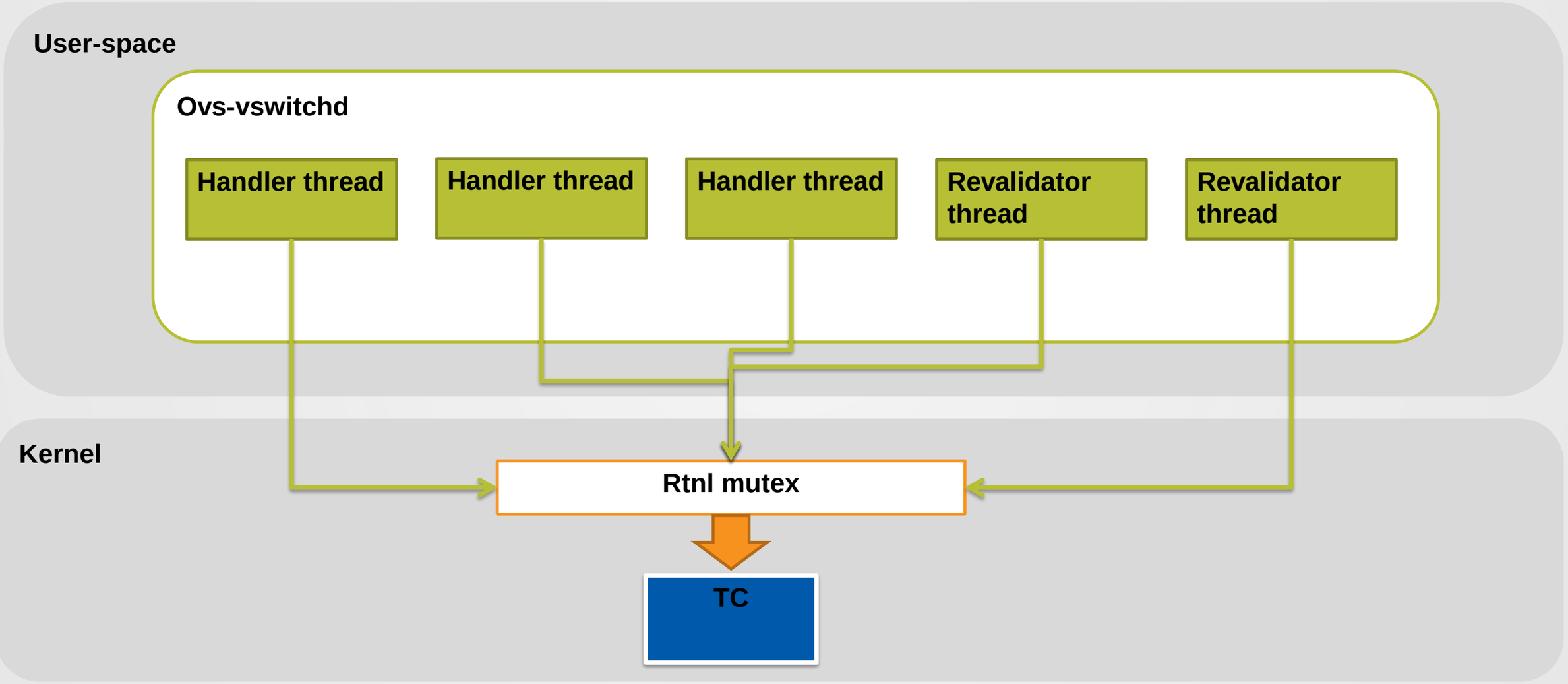
© 2018 Mellanox Technologies

Multi-threaded user space networking software

- Open vSwitch - a soft switch running within the hypervisor, or as the control stack for switching silicon.
- Uses TC kernel infrastructure for hardware offloads.
- Open vSwitch is multithreaded user-space application that can be configured to spawn arbitrary number of worker threads.
- TC kernel infrastructure is synchronized by One Big Lock(rtnl lock) that protects all netlink subsystem.

© 2018 Mellanox Technologies

Open vSwitch use-case overview



© 2018 Mellanox Technologies

Kernel TC architecture

© 2018 Mellanox Technologies

Problem with rtnl lock

- Rtnl mutex is used to synchronize netlink subsystem.
- Seemingly unrelated task like changing IP address of interface and creating/deleting TC rules excludes each other.
- Lock can be held for seconds(yes, seconds!) by some heavy operations:

```
gact$ time ls insert_sw* | xargs -n 1 -P 100 sudo tc -b

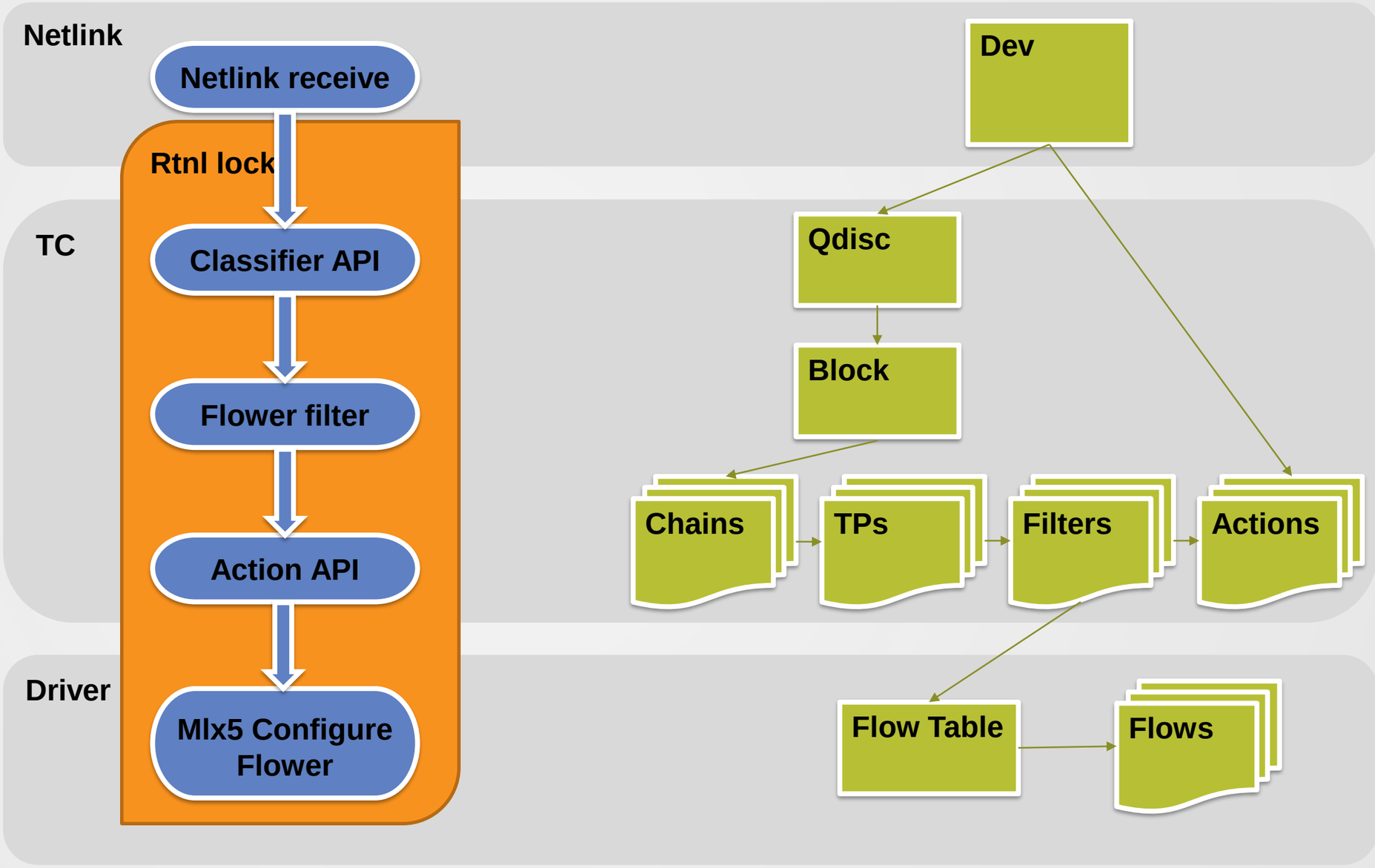
real    1m1.380s
user    0m40.090s
sys     9m17.291s
gact$ time sudo tc qdisc del dev eth0_0 ingress

real    0m1.817s
user    0m0.013s
sys     0m1.761s
gact$
```

© 2018 Mellanox Technologies

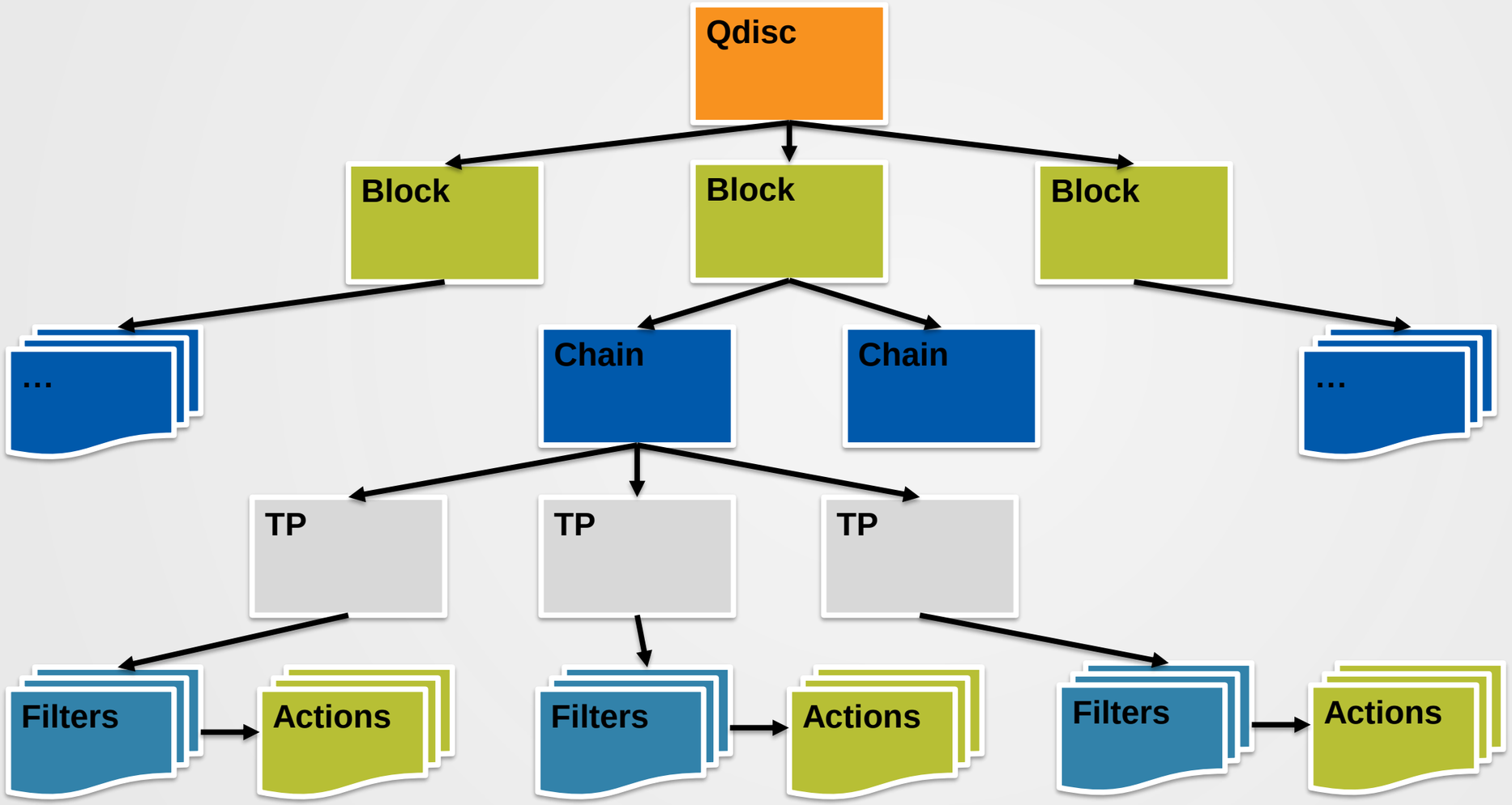
Current

- Global rtnl lock obtained early in call chain.
- No parallelism is possible.
- Classifier and Action API are protected from any network/netlink layer state changes with global rtnl lock
- All kinds of parallel-unsafe data structures are used (idr, linked list, non-atomic reference counters)



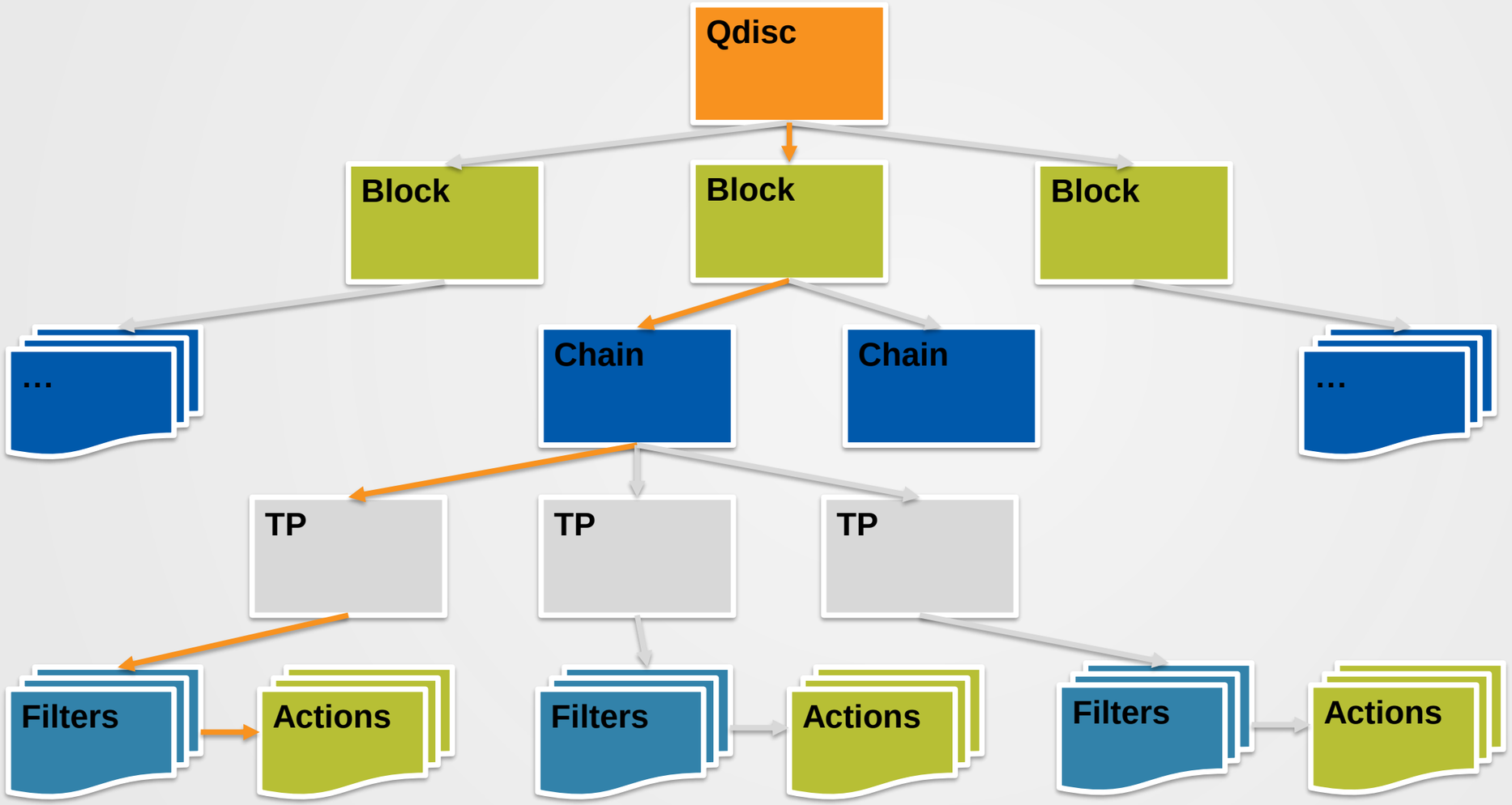
© 2018 Mellanox Technologies

TC cls-act data structure overview



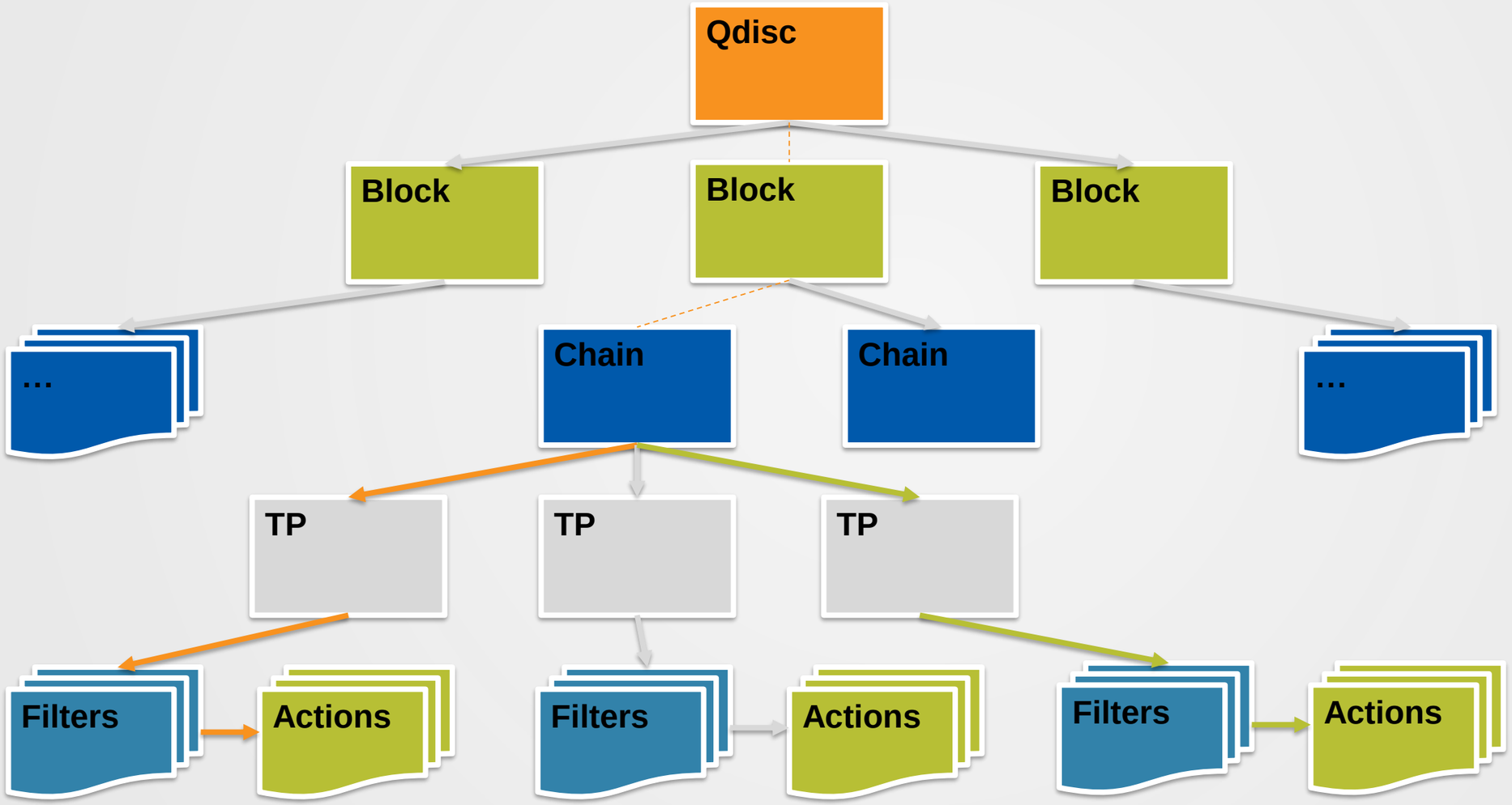
© 2018 Mellanox Technologies

TC cls-act data structure overview



© 2018 Mellanox Technologies

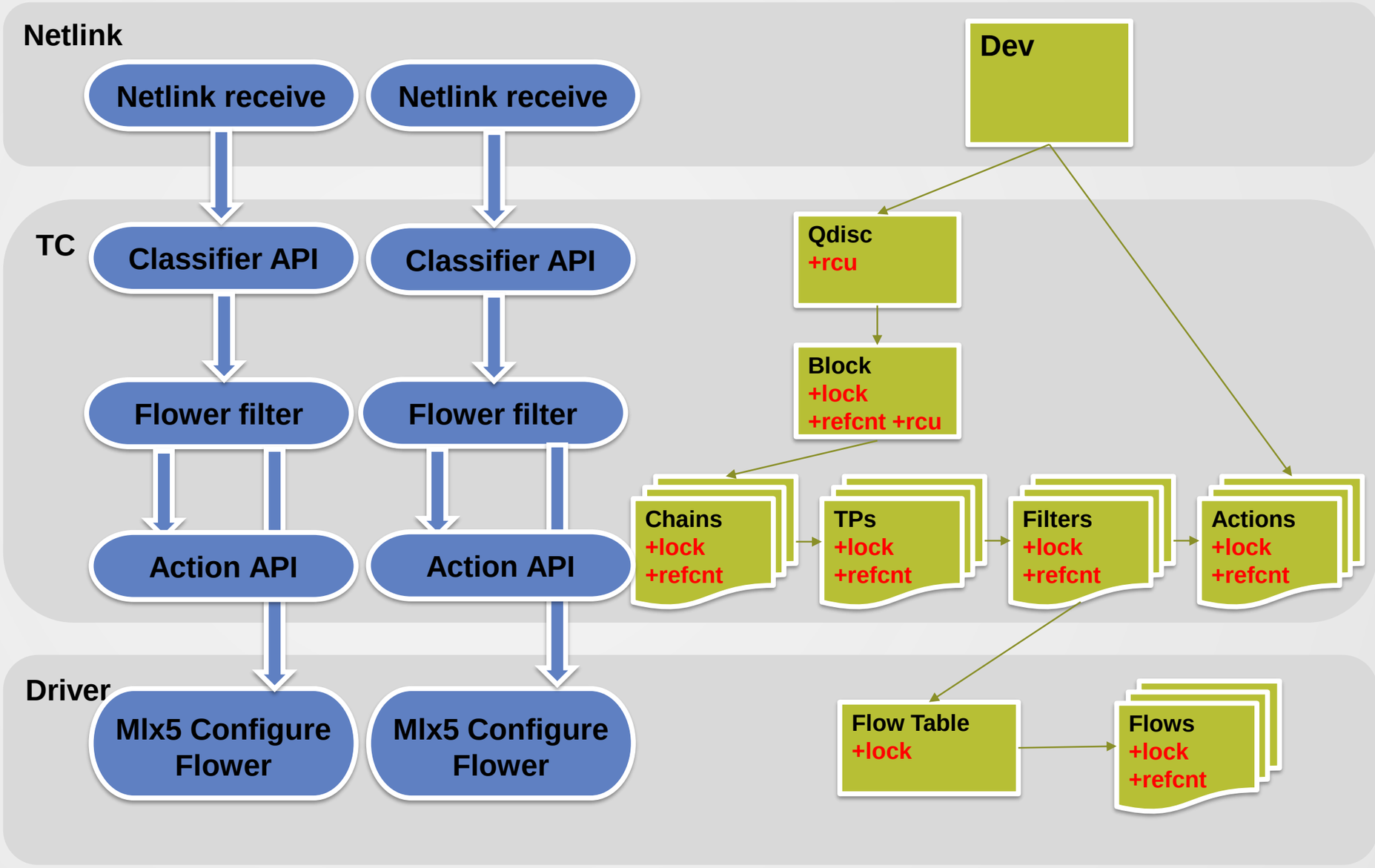
TC cls-act data structure overview



© 2018 Mellanox Technologies

Unlocked rules update path

- Rtnl lock is not used at all if qdisc, classifier and driver support unlocked execution.
- TC rules can be updated in parallel from user space.
- Data structure used in cls API and below it are protected with locks and extended with reference counters.
- Code refactoring to allow parallel filters insertion.
- Without protection provided by global rtnl lock this implementation must account for concurrent access through other TC APIs (qdisc, action, etc.) and in driver (neigh update, stats update, etc.)



© 2018 Mellanox Technologies

Development status

- Multiple different patchsets that refactors qdisc, classifier, action, hw offload layers to allow inserting rules in parallel.
- Unlocked implementation of ingress qdisc and flower classifier that are used by Open vSwitch.
- ~60 patches in total.
- Internal review -> done.
- Some patches have already been accepted upstream.
- Review in upstream mailing list -> in progress.

© 2018 Mellanox Technologies

Performance impact

© 2018 Mellanox Technologies

Benchmark description

```
netdev_conf_report24$ head -n 3 insert_sw_1_1
filter add dev eth0 protocol ip ingress chain 1 prio 10 handle 1 flower skip_hw dst_mac e4:11:22:33:44:50 src_mac e4:12:22:33:44:50 src_ip 192.168.111.1 dst_ip 192.168.111.2 ip_proto udp dst_port 1 src_port 1 action gact drop
filter add dev eth0 protocol ip ingress chain 1 prio 10 handle 2 flower skip_hw dst_mac e4:11:22:33:44:50 src_mac e4:12:22:33:44:50 src_ip 192.168.111.1 dst_ip 192.168.111.2 ip_proto udp dst_port 2 src_port 1 action gact drop
filter add dev eth0 protocol ip ingress chain 1 prio 10 handle 3 flower skip_hw dst_mac e4:11:22:33:44:50 src_mac e4:12:22:33:44:50 src_ip 192.168.111.1 dst_ip 192.168.111.2 ip_proto udp dst_port 3 src_port 1 action gact drop
netdev_conf_report24$ time ls insert_sw* | xargs -n 1 -P 100 sudo tc -b ^C
netdev_conf_report24$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Byte Order:            Little Endian
CPU(s):                24
On-line CPU(s) list:   0-23
Thread(s) per core:    2
Core(s) per socket:    6
Socket(s):             2
NUMA node(s):          2
Vendor ID:             GenuineIntel
CPU family:            6
Model:                 63
Model name:            Intel(R) Xeon(R) CPU E5-2620 v3 @ 2.40GHz
Stepping:              2
CPU MHz:               1200.424
CPU max MHz:           3200.0000
CPU min MHz:           1200.0000
BogoMIPS:              4800.11
Virtualization:        VT-x
L1d cache:             32K
L1i cache:             32K
L2 cache:              256K
L3 cache:              15360K
NUMA node0 CPU(s):     0-5,12-17
NUMA node1 CPU(s):     6-11,18-23
Flags:                 fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe syscall nx pdpe1gb rdtscp lm constant_tsc arch_perfmon pebs bts rep_good nopl xtopology nonstop_
tsc cpuid aperfmperf pni pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 ssse3 sdbg fma cx16 xtpr pdcm pcid dca sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand lahf_lm abm cpuid_fault epb invpcid_single pti i
ntel_ppin tpr_shadow vnmi flexpriority ept vpid fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid cqm xsaveopt cqm_llc cqm_occup_llc dtherm ida arat pln pts
```

© 2018 Mellanox Technologies

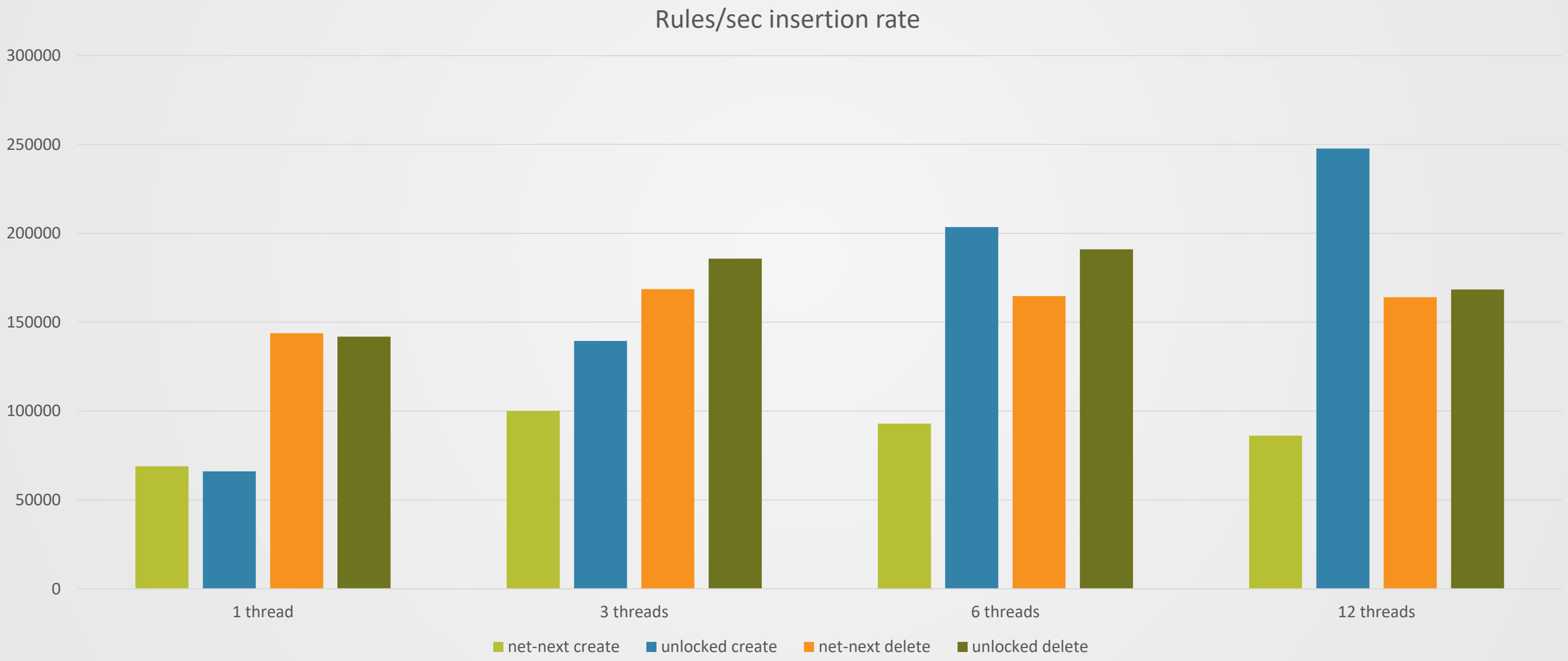
Methodology

- Methodology – insert 2.4 million of rules with N parallel tc instances.
- Run parallel tc instances with xargs. (no hardcoded, staged or otherwise “magic” tests that are unreproducible)
- Use cases tested:
  - Insert to same tc to test in worst case conditions by provoking maximum possible lock contention.
  - Insert to different chains in order to minimize lock contention.
  - Verify impact of using single action for all rules.
- Use flower classifier(L2+5-tuple) and simple gact drop action:

```
netdev_conf_report24$ head -n 3 insert_shared_multi_12_1
filter add dev eth0 protocol ip ingress chain 1 prio 10 handle 1 flower skip_hw dst_mac e4:11:22:33:44:50 src_mac e4:12:22:33:44:50 src_ip 192.168.111.1 dst_ip 192.168.111.2 ip_proto udp dst_port 1 src_port 1 action gact drop index 1
filter add dev eth0 protocol ip ingress chain 1 prio 10 handle 2 flower skip_hw dst_mac e4:11:22:33:44:50 src_mac e4:12:22:33:44:50 src_ip 192.168.111.1 dst_ip 192.168.111.2 ip_proto udp dst_port 2 src_port 1 action gact drop index 1
filter add dev eth0 protocol ip ingress chain 1 prio 10 handle 3 flower skip_hw dst_mac e4:11:22:33:44:50 src_mac e4:12:22:33:44:50 src_ip 192.168.111.1 dst_ip 192.168.111.2 ip_proto udp dst_port 3 src_port 1 action gact drop index 1
netdev_conf_report24$ head -n 3 insert_shared_multi_12_2
filter add dev eth0 protocol ip ingress chain 2 prio 10 handle 200001 flower skip_hw dst_mac e4:11:22:33:44:50 src_mac e4:12:22:33:44:50 src_ip 192.168.111.1 dst_ip 192.168.111.2 ip_proto udp dst_port 1 src_port 5 action gact drop index 1
filter add dev eth0 protocol ip ingress chain 2 prio 10 handle 200002 flower skip_hw dst_mac e4:11:22:33:44:50 src_mac e4:12:22:33:44:50 src_ip 192.168.111.1 dst_ip 192.168.111.2 ip_proto udp dst_port 2 src_port 5 action gact drop index 1
filter add dev eth0 protocol ip ingress chain 2 prio 10 handle 200003 flower skip_hw dst_mac e4:11:22:33:44:50 src_mac e4:12:22:33:44:50 src_ip 192.168.111.1 dst_ip 192.168.111.2 ip_proto udp dst_port 3 src_port 5 action gact drop index 1
```

© 2018 Mellanox Technologies

Worst case scenario – inserting to same tp



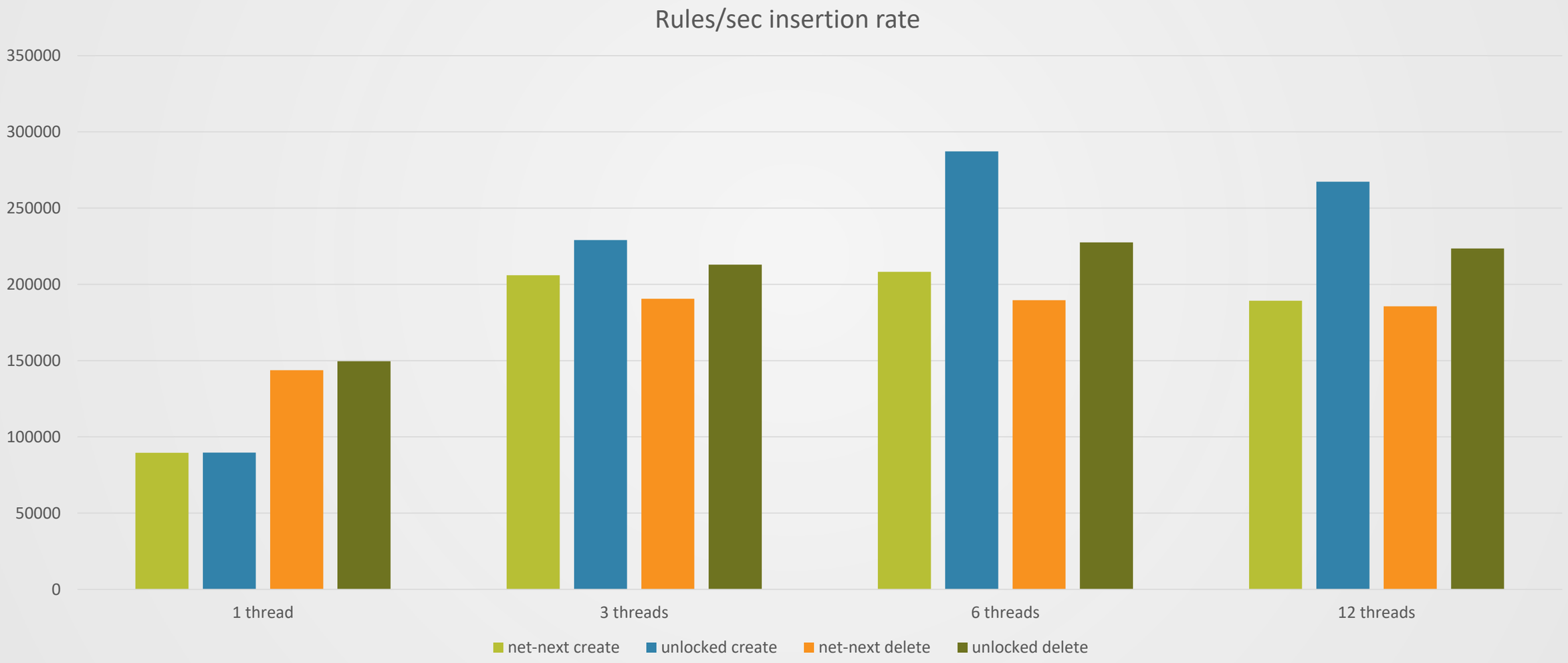
© 2018 Mellanox Technologies

Profiling data

```
Samples: 2K of event 'cycles', Event count (approx.): 6311703798
Overhead  Command  Shared Object     Symbol
  86.41%  tc       [kernel.vmlinux]  [k] queued_spin_lock_slowpath
   1.77%  tc       libc-2.23.so      [.] __memset_avx2
   0.86%  tc       [kernel.vmlinux]  [k] __gnet_stats_copy_basic
   0.78%  tc       [kernel.vmlinux]  [k] __gnet_stats_copy_queue_cpu
   0.73%  tc       tc                [.] tc_filter_modify
   0.49%  tc       libc-2.23.so      [.] __strcasecmp_l_avx
   0.45%  tc       [kernel.vmlinux]  [k] memcpy_erms
   0.45%  tc       [kernel.vmlinux]  [k] _raw_spin_lock
   0.30%  tc       [kernel.vmlinux]  [k] memset_erms
   0.29%  tc       [cls_flower]      [k] fl_delete
   0.29%  tc       [cls_flower]      [k] fl_dump
   0.21%  tc       tc                [.] matches
   0.21%  tc       [kernel.vmlinux]  [k] apic_timer_interrupt
   0.20%  tc       libc-2.23.so      [.] strlen
   0.20%  tc       libc-2.23.so      [.] __GI_____strtoull_l_internal
   0.16%  tc       libc-2.23.so      [.] __strspn_sse42
   0.16%  tc       [kernel.vmlinux]  [k] __kmalloc_node_track_caller
   0.16%  tc       [kernel.vmlinux]  [k] kfree
   0.16%  tc       tc                [.] batch
   0.16%  tc       [kernel.vmlinux]  [k] tcf_fill_node
   0.12%  tc       [kernel.vmlinux]  [k] syscall_return_via_sysret
   0.12%  tc       [cls_flower]      [k] fl_dump_key_val
   0.12%  tc       [kernel.vmlinux]  [k] __list_del_entry_valid
   0.12%  tc       libc-2.23.so      [.] __strcspn_sse42
   0.12%  tc       [kernel.vmlinux]  [k] qdisc_match_from_root
   0.12%  tc       [kernel.vmlinux]  [k] __tcf_proto_put
   0.12%  tc       [kernel.vmlinux]  [k] find_next_bit
   0.12%  tc       [kernel.vmlinux]  [k] tcf_action_dump
   0.12%  tc       [kernel.vmlinux]  [k] _raw_spin_lock_irqsave
   0.09%  tc       [kernel.vmlinux]  [k] nla_put
   0.08%  tc       [kernel.vmlinux]  [k] ktime_get
   0.08%  tc       [kernel.vmlinux]  [k] tcf_chain_tp_find
   0.08%  tc       [kernel.vmlinux]  [k] tcf_exts_dump
   0.08%  tc       [kernel.vmlinux]  [k] update_vsyscall
   0.08%  tc       [unknown]         [k] 0xfffffe0000216000
   0.08%  tc       [kernel.vmlinux]  [k] _raw_read_lock
   0.08%  tc       [kernel.vmlinux]  [k] timekeeping_update
   0.08%  tc       [cls_flower]      [k] fl_get
   0.08%  tc       [kernel.vmlinux]  [k] __nla_reserve
   0.08%  tc       [kernel.vmlinux]  [k] cpumask_next
   0.08%  tc       [kernel.vmlinux]  [k] read_tsc
```

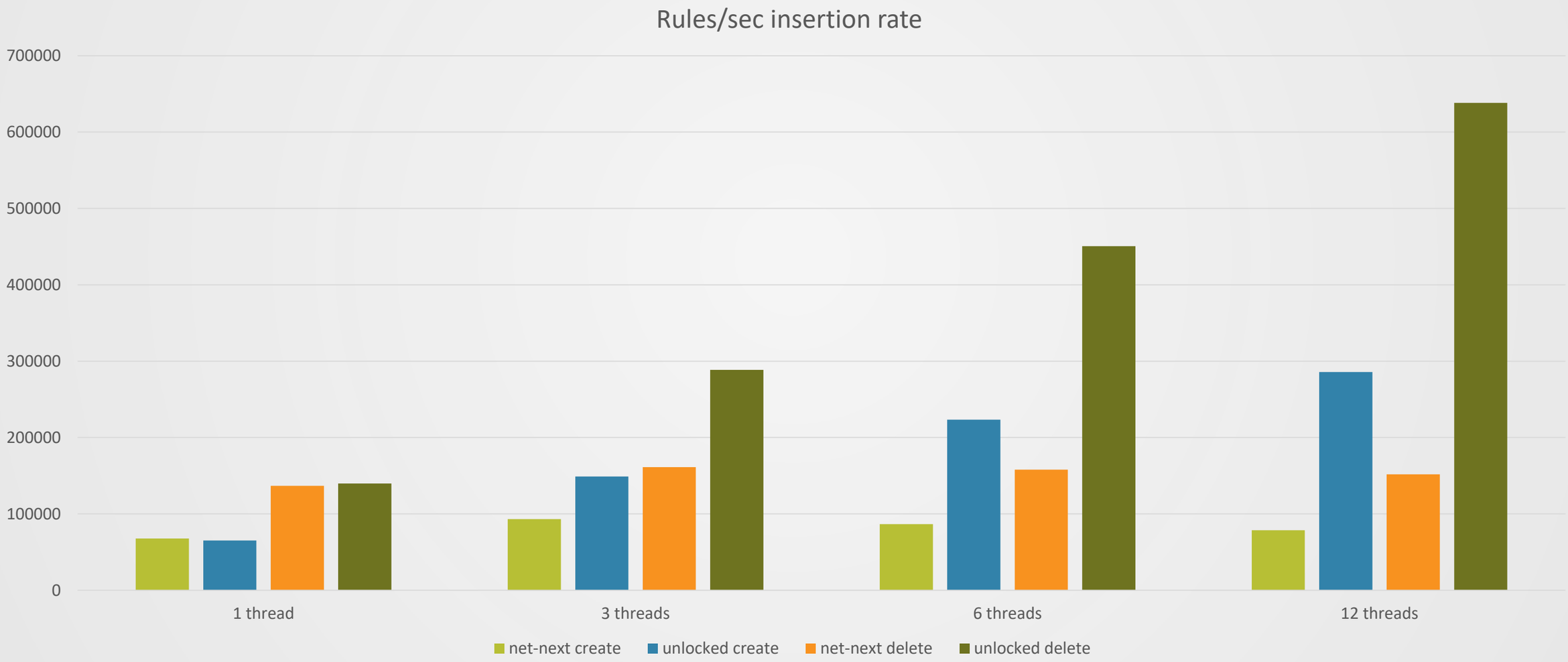
© 2018 Mellanox Technologies

Reducing lock contention by using shared action



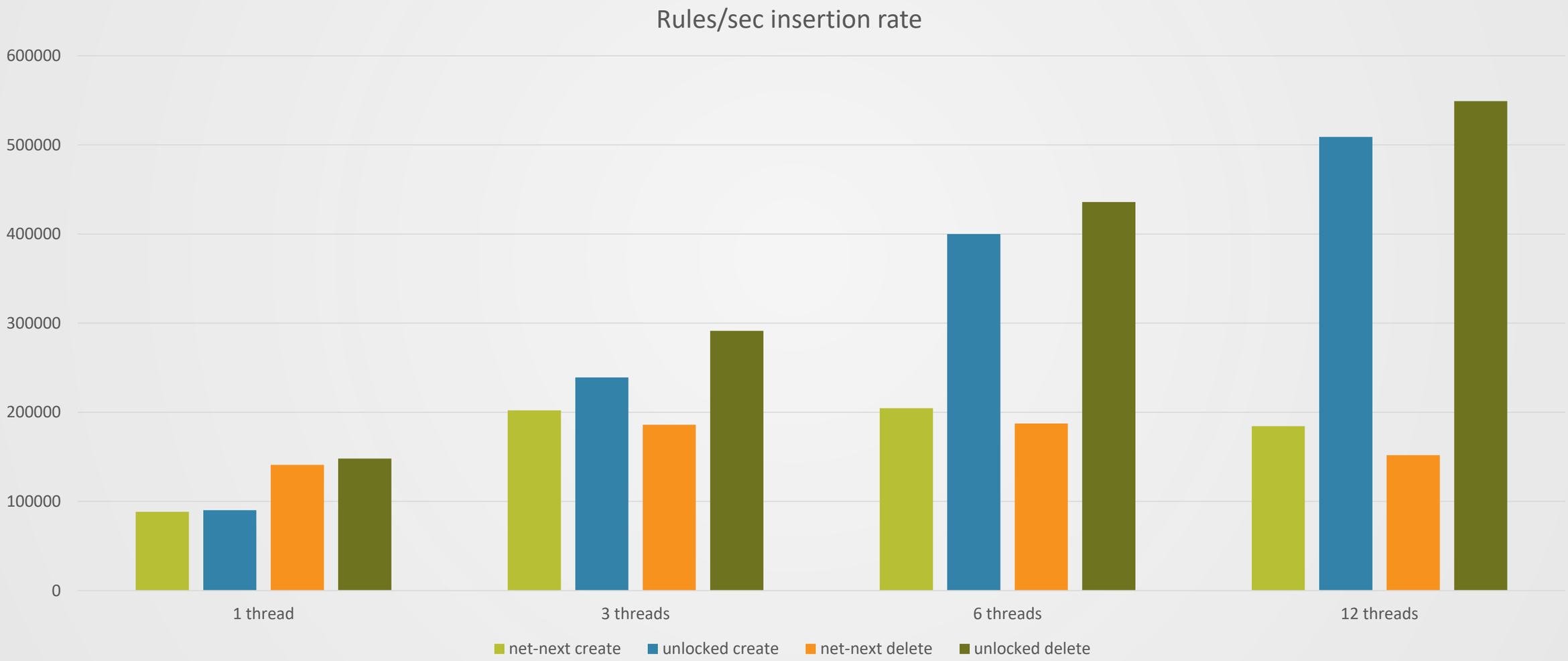
© 2018 Mellanox Technologies

Reducing lock contention making each tc instance create rules on different chain-tp



© 2018 Mellanox Technologies

Reducing lock contention – combining two previous approaches



© 2018 Mellanox Technologies

Conclusion

- Performance impact of introducing fine-grained locking without actual parallelism is negligible. (0-3% decrease depending on use case)
- Parallel execution provides performance benefits. (1.4x-3.6x)
- Scalability depends on use case. (less shared state -> less lock contention -> better scalability)

© 2018 Mellanox Technologies



Thank You

© 2018 Mellanox Technologies